

BUILDING LLM-POWERED APPLICATIONS · LECTURE 4

Frontend, Reactivity & SSE

A reactive UI that streams tokens from your backend as they're generated.

Press  for fullscreen · navigate with  

You ask an LLM app a question. Which feels faster to YOU as a user?

Spinner for 8s, then the whole 300-word answer appears at once

First words appear in 0.2s and keep flowing, total still 8s

By the end you can...

- Explain **reactivity**: change state → the UI updates itself
- `fetch` your Lecture 3 `/api/chat` backend from the browser
- Choose **SSE** over WebSockets for a one-way token stream
- Read & write the `data: ... \n\n` wire format, including the `[DONE]` sentinel
- Stream tokens into reactive state so the reply **types itself out**

Live example: the **reference app's frontend** (`web/app.js` at rag-demo.nat-d.uk) reads an SSE token stream and types the reply out as it arrives.

Where we're going

So far

- Lecture 3 gave us a `/api/chat` backend
- It can already `stream` tokens (Lecture 1)

Today

- What "reactivity" means
- Calling your API from the browser
- Streaming with Server-Sent Events

The goal

A chat box where the assistant's reply types itself out word by word — no spinner, no waiting for the whole answer.

In a SvelteKit `<script>` you write `let reply = $state("")` and the markup is `<p>{reply}</p>`. Later, in a loop, you run `reply += token` ten times. What does the user see?

Nothing until you call a `render()` or `update()` method by hand

The `<p>` re-renders on each `+=`, so the text grows token by token on its own ✓

Only the final value, because the DOM updates once when the function returns

An error — you can't reassign a `$state` variable after declaring it

What "reactivity" means

You change **state**; the UI updates **itself**. No manual DOM edits, no "find the element and set its text".

```
<script>
  let reply = $state("");    // a reactive variable
</script>

<!-- the markup re-renders whenever `reply` changes -->
<p>{reply}</p>
```

Append a token to `reply` and the `<p>` redraws on its own. That one idea is what makes streaming feel alive.

You keep chat turns in `let messages = $state([])`. A new reply arrives and you run `messages.push({ role: 'assistant', content: tok })`. The array is correct in the console, but the new bubble doesn't appear. Why?

Svelte re-renders on reassignment; `.push` mutates in place, so the `messages` binding never changes — use `messages = [...messages, x]` ✓

`$state` can only hold strings and numbers, never arrays

You must call a `render()` function by hand after every change

`{#each}` blocks can't display objects, only primitive values

One source of truth: `$derived` & `$effect`

Keep the **primitive facts** in `$state`; let the rest be **computed**. Don't juggle a pile of booleans by hand.

```
let prompt = $state("");
let streaming = $state(false);

// recomputes when either changes
let canSend = $derived(
  !streaming && prompt.trim() !== "");
```

`disabled={!canSend}` greys the button by itself
the instant `streaming` flips or the box empties.

```
// side-effect AFTER the DOM updates
$effect(() => {
  messages; // read → tracked
  log?.scrollTo({ top:
    log.scrollHeight });
});
```

`$effect` re-runs whenever state it *reads*
changes — here it pins the view to the bottom as
tokens land.

Calling your API from the browser

The browser talks to your Lecture 3 backend with `fetch`. Bind an input to state, POST it, and update state with the reply:

```
async function send() {  
  const messages = [{ role: "user", content: prompt }]; // history list  
  const res = await fetch("/api/chat", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ messages }),  
  });  
  const data = await res.json();  
  reply = data.answer; // reactive - UI updates  
}
```

This waits for the **whole** reply. For a long answer the user stares at a spinner. That's the streaming problem.

From the browser you call ``await fetch("/api/chat", { method: "POST", body: JSON.stringify({ messages }) })`` against your Lecture 3 backend, then ``await res.json()``. While the model is still generating its reply, what has the browser received?

Nothing yet — ``res.json()`` resolves only once the whole response body has arrived ✓

Each token as a separate ``fetch`` resolution, one promise per token

A partial JSON object that grows on every keystroke of the model

Think · pair · share

The `await res.json()` version and the streaming version take the **same 8 seconds** end to end. Why does streaming still feel dramatically faster — and when would it *not* help at all?

Talk it through

- What does the user actually **do** in those 8 seconds?
- Which number dropped — total time, or time-to-first-token?

Push further

- For a 3-word answer, is streaming worth the extra plumbing?
- What new ability does streaming unlock that buffering can't?

Streaming fixes **perceived** latency, not throughput — and it's what makes a **Stop** button possible.

Why streaming feels faster

Same total time — streaming gives the reader something at 0.4s, not 8s



Same total time — but streaming gives the reader the first token at ~0.4s instead of a blank screen for 8s.

The streaming problem

Wait for everything

A 300-token answer might take 8 seconds. The user sees a spinner the entire time, then the whole wall of text appears at once.

Stream as you go

The model already produces tokens one at a time. Forward each one the instant it's ready — text appears in ~200 ms and keeps flowing.

Same total time, but **perceived** latency drops massively — and the user can read (and stop) early. We need a wire that pushes chunks *after* the response has started.

You want `gemma-4-E4B-it`'s reply to appear in the browser word-by-word as it's generated, over ordinary HTTP, with the browser sending nothing back mid-reply. Predict: which approach can push each token to the browser the instant the server has it?

A normal `await res.json()` fetch — it already streams the body as the model types

The server keeps the response open and emits `data:` frames as tokens arrive; the client reads them as they land ✓

Only a WebSocket can deliver data before the response finishes

Poll the server with a fresh `fetch` every 100ms and diff what changed

SSE vs WebSockets vs plain fetch

Plain fetch

One request, one whole response.
Simple, but no early output.

SSE (this lecture)

Server → client, one-way stream over ordinary HTTP. Perfect for token streaming.

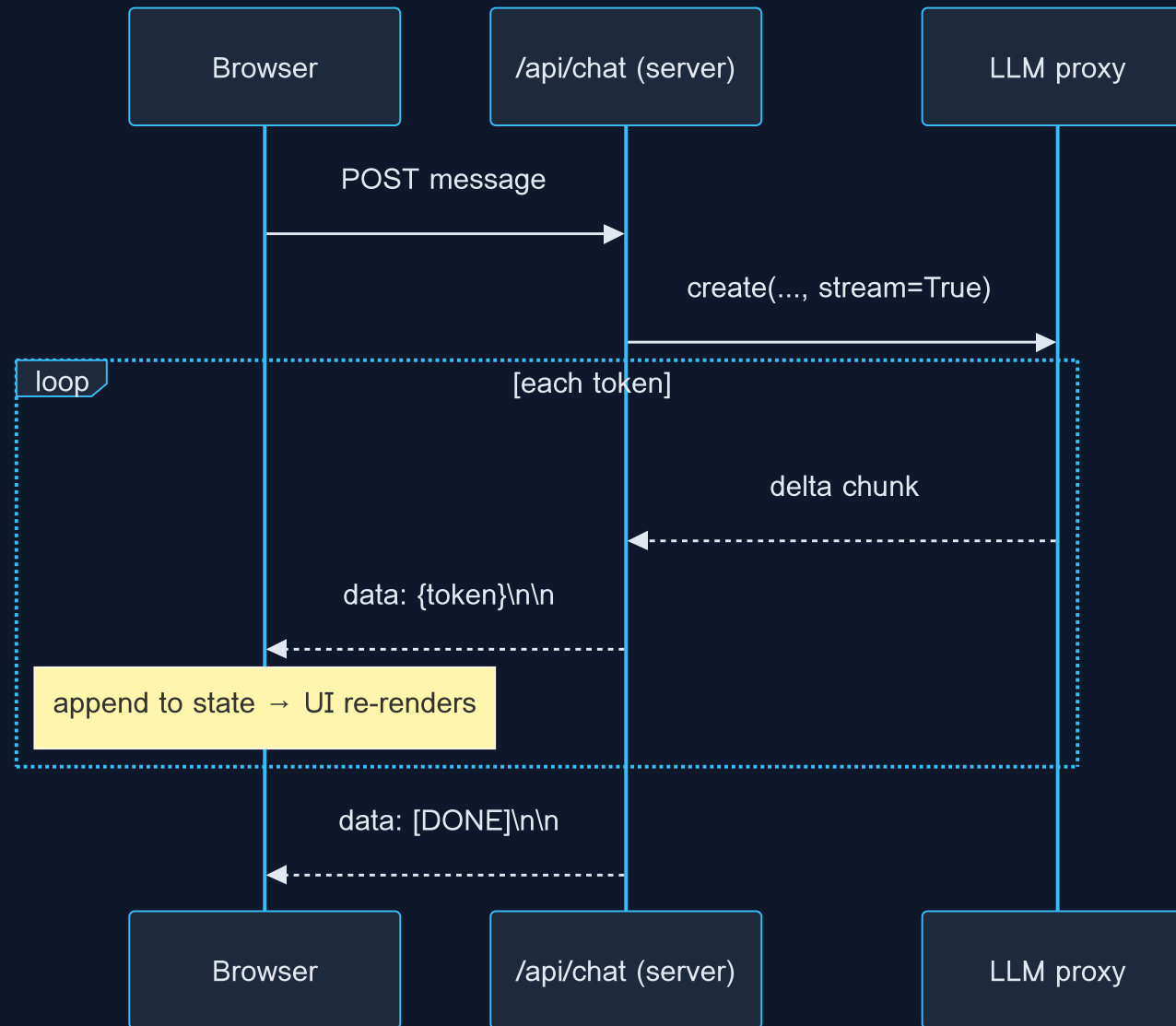
WebSockets

Two-way, full-duplex. Needed for chat rooms, multiplayer, live cursors.
Heavier: a separate protocol & connection to manage.

For "the server pushes the LLM's reply at me", one-way is all we need — so reach for the simpler tool: SSE.

Caveat: **auto-reconnect** is a feature of the browser's `EventSource`. The POST `fetch`-reader this lecture uses does **not** auto-reconnect.

Sse Flow



Your only requirement: the server pushes the LLM's reply to the browser token-by-token. The browser sends nothing back mid-stream. Best transport?

WebSockets — streaming always needs a two-way (both directions at once) protocol

SSE — one-way server → client over ordinary HTTP is exactly enough ✓

Plain fetch + JSON — just make the request and await the whole body

Long-polling — fire a new fetch every 100ms and compare what changed

The `text/event-stream` format

SSE is plain text over HTTP. Each event is a `data:` line, terminated by a blank line (`\n\n`):

```
# header: Content-Type: text/event-stream

data: Hello

data:  there

data: [DONE]
```

The browser reads one event per `\n\n`; the payload after `data:` is a raw token (or JSON you parse), and `[DONE]` signals the end. Real tokens are **sub-word pieces** (`"Ray"`, `"leigh"`) you *concatenate* — not space-joined.

The newline trap → wrap tokens in JSON

Bare `data: {tok}\n\n` works only while a token has **no newline**. But real replies emit `"\n"` tokens (paragraphs, list items). A newline token becomes:

```
data: ← the token IS "\n", so this line is "data: \n\n\n"
```

```
# ↑ a blank line landed mid-token → frame boundary corrupted
```

Fix: make every payload a **single safe line** by JSON-encoding it. Server:

```
import json
yield f"data: {json.dumps({'token': tok})}\n\n" # client: JSON.parse(f.slice(6)).token
```

Use `json.dumps`, not Python's `{tok!r}` repr — repr emits single-quoted, **invalid JSON**. This is exactly **Practice #2**.

Your server yields raw ``data: {tok}\n\n``. It works on short answers, then you ask for a bulleted list and the stream corrupts. The model emitted a token that is literally ``"\n"``. What went wrong?

The model is broken — it should never produce newline tokens

A newline inside the payload made a blank line, which the client reads as the early end of a frame;
JSON-encode each token to keep it on one line ✓

FastAPI strips newlines from yielded strings, dropping the token

The client's ``TextDecoderStream`` can't decode newline characters

The streaming demo prints one ``data:`` line per token, then ends.
How does the CLIENT know there are no more tokens?

The HTTP connection closes and `read()` returns done — that's the only signal

A final ``data: [DONE]`` marker line the client watches for ✓

An empty ``data:`` line with no payload

It counts tokens against a length sent in the first frame

Server Yields

```
# What your server SENDS over the wire: one SSE "data:" frame per token.
# Tokens are sub-word pieces ("Ray", "leigh"), exactly as the browser reads
# them off /api/chat. In FastAPI you'd `yield` these lines instead of print().
#
# A real server reads the tokens LIVE from the model:
#     for chunk in client.chat.completions.create(..., stream=True):
#         token = chunk.choices[0].delta.content or ""
# We hardcode them here so the FRAME FORMAT is the whole focus and the run is
# the same every time. (The sandbox buffers stdout, so it can't show tokens
# arriving live – the "Live stream – the real thing" slide does that for real.)

tokens = ["Ray", "leigh", " scattering", " makes", " the", " sky", " blue", "."]

for token in tokens:
    # print() ADDS its own trailing "\n", so f"data: {token}\n" emits
    # `data: {token}` + "\n" (ours) + "\n" (print's) = "\n\n" = ONE SSE frame.
    print(f"data: {token}\n", flush=True)
```

Parse Sse Bytes

```
# CODE-ALONG: the buffer-and-split logic your BROWSER runs, in plain Python.
# Network chunks do NOT line up with SSE frames – one read() can hand you half a
# frame, or three at once. We stitch with a buffer and split on the blank line.

# Pretend these are raw byte-chunks as they arrive off the socket – note the
# splits fall in the MIDDLE of frames, not on frame boundaries:
chunks = [
    b"data: Ray\n\nda",          # frame 1 complete, frame 2 starts
    b"ta: leigh\n\ndata:  sc",    # frame 2 complete, frame 3 starts
    b"attering\n\ndata: [DONE]\n\n",
]

buffer = ""
reply = ""
for chunk in chunks:
    buffer += chunk.decode()      # like reader.read() + TextDecoderStream
    frames = buffer.split("\n\n")
    buffer = frames.pop()         # keep the last, possibly-partial frame
    for f in frames:
```

Live stream — the real thing

System: You are a concise teacher. Answer in a few short, clear sentences.

Prompt

```
Why is the sky blue? Answer in 3 short sentences.
```

(live streaming demo — run it in the deck)

Server side: yield the chunks

In FastAPI, return a `StreamingResponse` over an async generator that **yields** SSE frames as the LLM produces them:

```
async def token_stream(msg):    # client = AsyncOpenAI()
    stream = await client.chat.completions.create(
        model="gemma-4-E4B-it", messages=msg, stream=True)
    async for chunk in stream:
        tok = chunk.choices[0].delta.content or ""
        if tok:
            yield f"data: {tok}\n\n"
    yield "data: [DONE]\n\n"

@router.post("/api/chat")
async def chat(body: ChatIn):
    return StreamingResponse(token_stream(body.messages),
        media_type="text/event-stream")
```

Why `delta.content or ""`?

Not every streamed chunk carries text. The **first** chunk announces the `role`; the **last** carries a `finish_reason`. On those, `delta.content` is `None`.

```
# chunk 1 - role only, no text
delta = { "role": "assistant" }           # .content is None
# chunk 2..n - the actual tokens
delta = { "content": "Ray" }              # "Ray", "leigh", " scattering"...
# last chunk - stop signal, no text
delta = {}; finish_reason = "stop"        # .content is None
```

`delta.content or ""` turns those `None`s into empty strings, and the `if token:` guard skips them cleanly — so you never emit a stray `data: None` frame for the metadata chunks.

Client side: read the stream

We POST, so we read the body's **stream** ourselves and append each token to reactive state. The UI types itself out:

```
const body = JSON.stringify({ messages }); // history list
const res = await fetch("/api/chat", { method: "POST", body });
const reader = res.body.pipeThrough(new TextDecoderStream()).getReader();
let buf = ""; // carry partial frames across reads
while (true) {
  const { value, done } = await reader.read();
  if (done) break;
  buf += value;
  const frames = buf.split("\n\n");
  buf = frames.pop(); // last piece may be incomplete - keep it
  for (const f of frames) {
```

Network chunks don't align with frames, so `buf` stitches partial reads back together. (`EventSource` does this for you, but it's **GET-only**.)

Why the buffer? Chunks $\neq$ frames

TCP hands you **bytes**, not events. A single `read()` can split a frame in half, or deliver several at once. Watch the buffer absorb a mid-frame split:

```
read() #1 → "data: Ray\n\ndata: le"
split("\n\n") → ["data: Ray", "data: le"]
frames.pop() → emit "Ray", keep buf = "data: le" ← partial!

read() #2 → buf + "igh\n\ndata: [DONE]\n\n"
           = "data: leigh\n\ndata: [DONE]\n\n"
split("\n\n") → ["data: leigh", "data: [DONE]", ""]
frames.pop() → emit "leigh", see [DONE], keep buf = ""
```

The trailing `frames.pop()` is the trick: after splitting, the **last element** is always either empty or a partial frame — stash it and prepend it to the next read.

The symptom: a newline token splits the frame

Ask for a 2-line list and the model emits a `"\n"` token. Bare `data: {token}\n\n` framing turns three tokens (`"a"`, `"\n"`, `"b"`) into:

```
data: a
```

```
data: ← the token IS a \n
```

```
data: b
```

The middle frame is `data:` + the token's own `\n` + the blank line = `data: \n\n`.

The client splits on `\n\n`, reads an **empty payload**, and the next split lands in the wrong place. One newline desynchronises the whole stream.

The fix: one token = one JSON line

Wrap every token with `json.dumps`. It escapes the newline to the two characters `\` + `n`, so the payload is **always one line**:

Server

```
import json
print(f"data: {json.dumps({'token': tok})}\n")
```

Use `json.dumps`, not `{tok!r}` — that emits single-quoted text that is **not valid JSON**.

On the wire

```
data: {"token": "a"}

data: {"token": "\n"}

data: {"token": "b"}
```

The newline is now **data inside a string**, not a frame boundary. Client: `JSON.parse` → `.token`.

Why not just use `EventSource`?

The browser ships a built-in SSE client that handles framing *and* reconnection for you:

```
const es = new EventSource("/api/chat?message=hi");
es.onmessage = (e) => { reply += e.data; }; // framing done for you
```

`EventSource` — when a GET fits

Auto-frames, auto-reconnects via `Last-Event-ID`. But: GET-only, no custom body.

`fetch-reader` — when you POST

A chat message is long and belongs in a POST body. You frame it yourself and don't get auto-reconnect.

Rule: `EventSource` for small GET inputs; the streamed `fetch` reader (this lecture) whenever you need a request body.

Buffer Misaligned Chunks

```
# Why the buffer is NOT optional. The network delivers bytes in whatever sizes
# it likes -- frame boundaries (\n\n) do NOT line up with read() boundaries.
# Here two "reads" carry three frames; read 1 ends mid-frame at `{"tok`.
import json

chunks = [
    'data: {"token": "Ray"}\n\ndata: {"tok',    # ends mid-frame!
    'en": "leigh"}\n\ndata: [DONE]\n\n',
]

buffer = ""
out = []
for chunk in chunks:                # each `chunk` = one reader.read()
    buffer += chunk
    frames = buffer.split("\n\n")
    buffer = frames.pop()           # last item may be partial -- carry it over
    for frame in frames:
        if not frame.startswith("data: "):
            continue
        payload = frame[len("data: "):]
        if payload == "[DONE]":
```


Error frames: don't trail off into silence

If the LLM throws **after** streaming starts, the status is gone — it was `200` at the first byte. You can't send a 500. Yield an **error frame**:

Server

```
try:
    async for chunk in stream:
        tok = chunk.choices[0].delta.content or ""
        if tok:
            yield f"data: {json.dumps({'token': tok})}\n\n"
except Exception as e:
    yield f"data: {json.dumps({'error': str(e)})}\n\n"
yield "data: [DONE]\n\n"  # always close cleanly
```

Client

```
const msg = JSON.parse(payload);
if (msg.error) {
    reply += `\n[stream error: ${msg.error}]`;
    return;
}
reply += msg.token;
```

The `[DONE]` after `except` lets the client end its loop the same way on success or failure — the spinner always clears.

Streaming: symptom → fix

Symptom	Likely cause	Fix
Text after a line break vanishes	bare <code>data: {token}</code> + a <code>\n</code> token	JSON-wrap each token
Random tokens missing (<code>leigh</code>)	parsing each <code>read()</code> alone, no buffer	buffer + keep trailing partial
<code>JSON.parse</code> throws on a frame	used <code>{token!r}</code> not <code>json.dumps</code>	emit real JSON
Answer trails off, no error shown	upstream threw after <code>200</code>	yield an <code>error</code> frame
Reconnect replays from the top	no <code>id:</code> / <code>Last-Event-ID</code>	number frames, resume after id
Dies during the "thinking" pause	idle connection reaped by a proxy	heartbeat <code>: comment</code> frame
Works local, clumps in prod	buffering proxy / gzip / CDN	opt out of buffering

The §7 reader is the skeleton; this is the muscle that keeps it standing when the model writes a newline or the network stalls.

Your stream uses bare ``data: {token}\n\n`` framing. It works on short replies, but as soon as you ask for a multi-line list, everything after the first line break disappears. What's the cause and the fix?

The proxy is buffering — set `X-Accel-Buffering: no` on the response

A token that is itself ``n`` makes the frame end early (``data: \n\n``); wrap each token in JSON (``data: {"token": ...}``) so the payload is always one line ✓

The client forgot to call `JSON.parse`; parse every frame and the newlines render

`TextDecoderStream` drops newline bytes; switch to reading `res.body` as an `ArrayBuffer`

Keep the conversation

One `reply` isn't a chat. Hold the **whole history** in a reactive array and grow the last turn token-by-token:

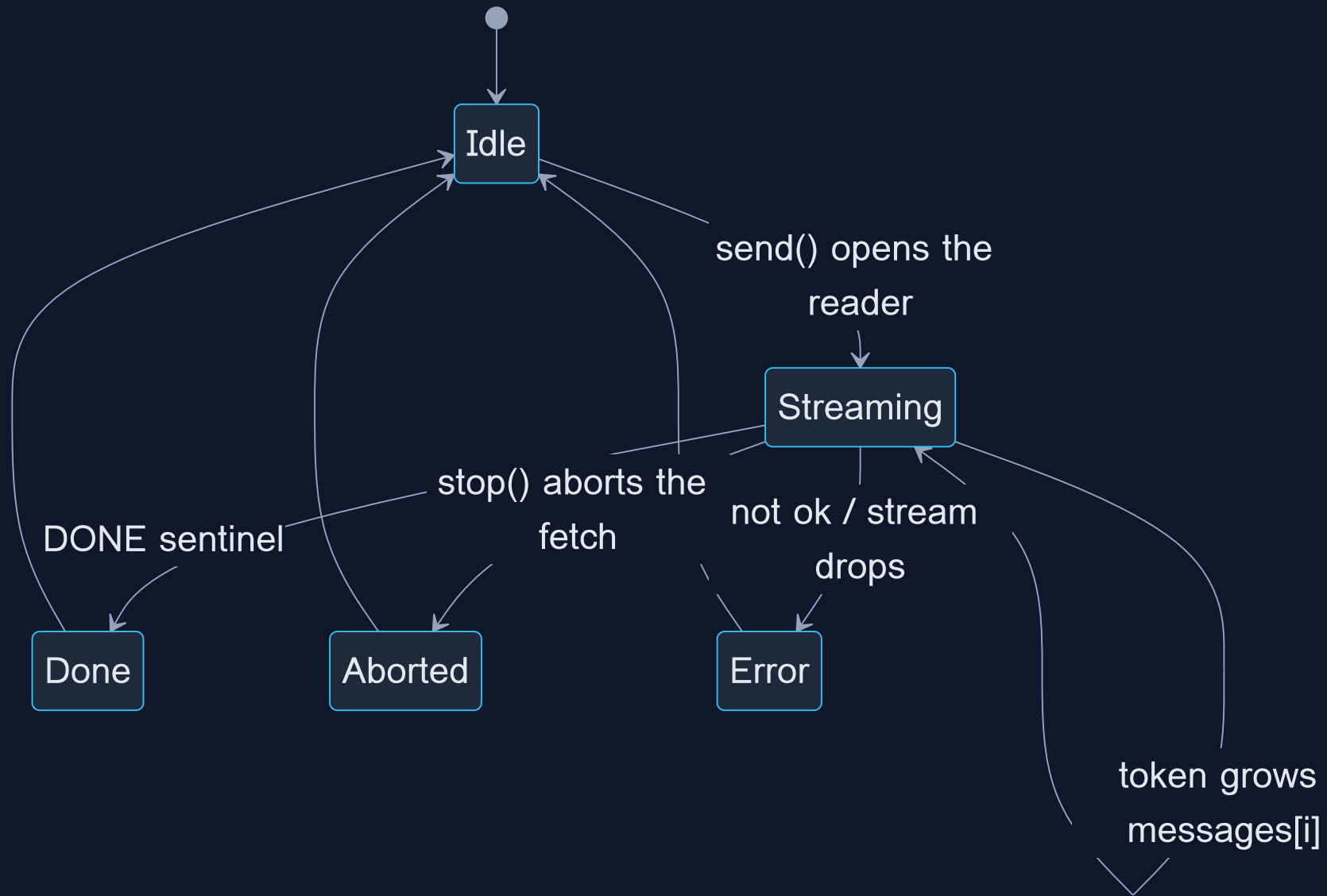
```
let messages = $state([]); // [{role, content}, ...]

// user turn - REASSIGN, don't .push()
messages = [...messages, { role: "user", content: prompt }];
// empty assistant turn we stream into
messages = [...messages, { role: "assistant", content: "" }];
const i = messages.length - 1;
```

```
{#each messages as m}
  <p class={m.role}>{m.content}</p>
{/each}
```

Send the **full** `messages` list every turn — what lets the model remember, and what **HW2** grades.

Ui States



Stopping early: cancel the stream

Streaming promised "the user can stop". Wire an `AbortController` into the `fetch`, then call `abort()` from a Stop button:

```
let controller = null;

async function send() {
  controller = new AbortController();
  const res = await fetch("/api/chat", {
    method: "POST", body,
    signal: controller.signal });
  // ...read loop...
}

function stop() { controller?.abort(); }
```

Stop paying for it

`abort()` closes the connection, which ends the server generator's loop — but only if the server notices.

```
# FastAPI, inside token_stream:
if await request.is_disconnected():
    break    # stop pulling tokens
```

Without the `is_disconnected()` check the proxy keeps generating (and billing) tokens nobody will ever read.

Handling errors: two failure points

`fetch` only rejects on a **network** failure — an HTTP 500 still *resolves*. So you must check `res.ok` yourself, and wrap the read loop:

```
const res = await fetch("/api/chat", { method: "POST", body, signal });
if (!res.ok) { reply = `Error: ${res.status}`; return; } // 4xx/5xx don't throw

try {
  // ...the buffered read loop...
} catch (err) {
  if (err.name !== "AbortError") // the Stop you caused on purpose
    reply += `\n[stream error: ${err.message}]`; // a real mid-stream drop
}
```

Server side: if the LLM throws mid-stream, yield a frame the client recognises — `data: {"error": "upstream failed"}\n\n` — instead of letting the connection die silently.

Your client does `reply += value` on every `reader.read()` chunk instead of splitting on `\n\n`. The model streams fine. What breaks?

Nothing — chunks always arrive as exactly one complete frame

The literal text ``data: `` and ``[DONE]`` show up in the visible reply, and a frame split across two reads is broken ✓

The connection never closes, so the request hangs forever

The browser refuses the response without an ``EventSource``

"It streams on localhost, clumps in prod"

The classic deployment surprise: works locally, then in production the whole reply lands **all at once** again. The culprit is almost always a **buffering reverse proxy** between server and browser.

Why it happens

- nginx / ingress buffer responses, flushing only a big block at a time
- gzip layers coalesce small frames to fill their window
- CDNs (Cloudflare) can re-buffer chunks

This very app runs behind Cloudflare + an ingress. If your stream works locally but clumps deployed, **suspect the proxy first** — not your code.

The opt-out

```
# response header
X-Accel-Buffering: no
```

...and keep **no compression** in front of
`text/event-stream`.

Your SSE chat types itself out perfectly on `localhost`, but once deployed behind nginx + Cloudflare the reply appears all at once after a pause. Most likely cause?

Your `token\_stream` generator is buffering — add more `yield`s

A proxy/CDN in front is buffering the response; set `X-Accel-Buffering: no` and keep compression off the event-stream ✓

The browser silently disabled streaming because it's not `localhost`

`gemma-4-E4B-it` only streams over unencrypted HTTP, not HTTPS

Recap & what's next

You can now

- Explain reactivity: state → UI updates
- Call `/api/chat` from the browser
- Pick SSE over WebSockets for one-way streams
- Read & write the `data:` wire format
- Keep & render the chat history reactively

Read the lecture notes, then continue HW2 — turn your backend into a streaming web UI.

Next lecture

Lecture 5

Building a Vector Storage — embed documents as vectors and run nearest-neighbor semantic search, the foundation for RAG.

